FIT5226 Project

Stage 3 - Multi-agent learning of a coordination problem (Q-Learning and Game Theory)

This document describes Stage 3 of the FIT 5226 project, its final stage. It builds on the previous stages which were conducted as group projects. You are allowed to use the results of all work that you participated in with your assigned group for Stage 3 but not those of any other group or work that you didn't participate in. You are also not allowed to consult about the individual phase of Stage 3 (see below) with other members of your group.

Assessment	Topic	Release	Due (Sunday)	Weight	Assessment mode
in-semester	Project Stage 1 (Table-based RL, single agent)	Wk3	Wk 5	10%	Group
in-semester	Project Stage 2 (Deep RL, single agent)	Wk 5	Wk 8	10%	Group
in-semester	Project Stage 3 (MARL)	Wk 9	Wk 12	55%	Mixed (comprising an individually conducted and assessed implementation and a group report giving a comparative evaluation of the team members solutions)

This project and the mid semester quiz constitute your *entire* assessment for the unit. There is no exam.

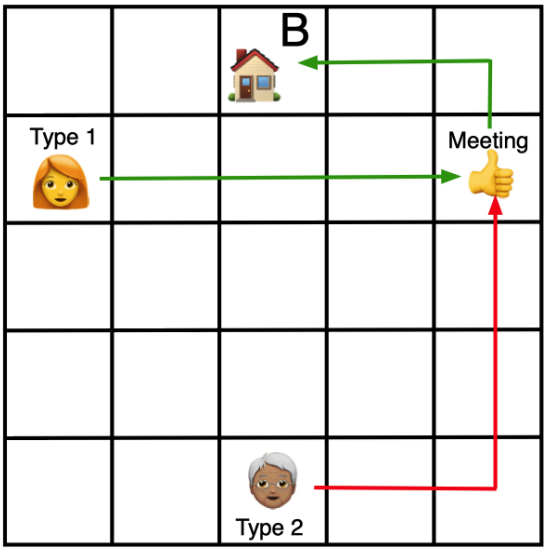
Phase 3 (the current Phase) concerns *Multi-Agent Learning*. The majority of this is conducted and marked as an individual assignment. The individual component (Stage 3.1) contributes 45% of the assessment. This is complemented with a group component (Stage 3.2), which contributes the remaining 10%.

“Give me six hours to chop down a tree and I will spend the first four sharpening the axe.”

— Commonly misattributed to Abraham Lincoln

Tasks for Stage 3.1 (Individual assessment)

You will write code for *multiple* agents in a square grid world to learn a simple transport task that requires collaboration between the agents. You are free to use Deep-Q or Tabular-Q as you see fit. The task now requires coordination between two agents. You will also use game theory to analyse the behaviour of these agents.



Your agents still have the same four actions that they can execute: “move north/south/west/east”. If you wish, you may add a “wait” action to this, in which the agent remains on the same grid square. Each agent starts at a random location. As before, an item needs to be transported to a location B and as in Stage 2, B is in a random cell of the grid in each episode (but never in the same cell as an agent). Each agent can only carry a single item at a time and they are in possession of this item from the start of the episode, ie. they don’t first need to pick it up.

There are now two types of agents: Both types can drop off at B. However, the drop-off location is now only accessible with a secret code! The tricky part is that each agent starts out knowing only one part of the secret: Type 1 knows the first part of the secret secret and Type 2 the other part.

As the full secret is required to access B and to be able to deliver, an agent must meet an agent of the other type before they can access location B and thus complete the mission. The two agents are required to exchange the information *before* they arrive at B. It is not possible to share the information at B (The guards at B will identify anyone as a spy, who doesn’t arrive in possession of the full secret!). To complete the mission, it is sufficient if only one of the two agents delivers an item at B.

Pick-up and drop-off are still executed automatically: no explicit action has to be executed for this. When an agent reaches B they automatically drop off the item that they carry provided they carry one. Information sharing when two agents of different types meet is also automatic.

To keep the learning time within limits we work on a small grid (5x5 as before) and we only use 2 agents of each type, ie. four agents in total.

To further speed up the learning, all agents *of the same type* are allowed to share the same DQN or Q-Table.

The agents may observe their own location, the location of B, and, of course, whether they are in possession of the full secret. Note that you can “buy” additional sensors to observe more information if you find this helpful (see below).

All agents normally have to perform their update sequentially *in a random order*.

Simplifying the task by “purchasing” options

You have the option to balance aiming for higher performing agents with the complexity of the problem structure. The performance that your agents reach influences the marks you will receive for your solution and you can “purchase” options to simplify the task structure so that it becomes easier to train high performing agents. The catch, however, is that you will have to “pay” for each option that you select with points. You can compensate for the “purchase price” by training a better performing agent that is rewarded with “performance points” or you can solve the unmodified task and settle for a lesser performance. The table below details the options available and their impact on marks received.

Your raw mark for the implementation categories “Solution and Functionality” and “Implementation and Coding” of the rubric will be scaled by a factor of
$\alpha = 1 - \frac{33}{200} max(0,\ C - B)$,
where *C* is the sum of costs for your chosen options and *B* the sum of points for the substantiated performance. In other words, for each 2 points of purchase cost that are not compensated for by outstanding learning performance you will lose ⅓ of the marks in this category.

Table 1: Options available

Option Type	Item	Description	Cost
Sensors	*All* agents of opposite type (*direction* to)	You may augments the observation space for all agents with the direction to all other agents of the opposite type (A list of 2d-vectors (otherPos-myPos))	5
	All agents of *opposite* type (location)	You may augments the observation space for all agents with the location of all other agents of the opposite type (A list of 2d-vectors (otherPos))	4
	Closest agent of opposite type (*direction* to)	As above but only for the closest agent instead of all agents	3

	Closest agent of opposite type (*location*)	As above but only for the closest agent instead of all agents	2
Coordination	Central clock	This allows you to coordinate the update schedule of your agents. Instead of having to update all agents in random order (as described above) you can update them in round-robin fashion or any other order that you determine.	1
Training conditions	Off-the-job training	Instead of having to learn with each episode starting from random locations for all agents and B you can define a schedule for the configuration at the start of training episodes and pick the agents to be trained in any round.	1
Setup	Fixed delivery location B	Instead of having to service a random delivery location, the target location B is always in the bottom right corner of the grid.	6

Table 2: Performance Points

You can receive up to 2 performance points for each category, so that a maximum total of 6 is possible. To receive these you must provide code that measures the claimed performance!

Performance level (percentage of scenarios solved in less than 15 steps)	Epochs required to train to this level[1]	Average excess path length[2]	Performance Points
>98%	<20,000	<0.5	2
>85%	<50,000	<2.5	1

[1] Note that we are measuring time in epochs instead of walltime; this is not a coding class!
[2] Excess over minimum possible path length. Calculate this carefully! It may not be as trivial as it appears at first sight.

Extended Deep-Q Library Code

As before, if you decide to use Deep-Q we do not expect you to be able to program the low-level handling and learning of the Q-network since neural network knowledge is not a prerequisite for the unit.

We are thus providing you again with a Python module that you can use at a higher level. This is available on Moodle as a Jupyter notebook **`Stage3Skeleton.ipynb`** in the Assignment section together with these instructions. It is a minor modification of the Stage 2 Skeleton so that two different DQNs can be used.

In contrast to Stages 1 and 2 you now have multiple agents, so the only sensible way to implement this is by using a class for the agents. Since you also need multiple DQNs (one combination of target and prediction network for each agent type) we have wrapped the DQN skeleton in a class as well. You can simply get new instances of this class to get multiple, independent DQN pairs. Since the DQN is associated with the agent class (all agents of the same class share the same DQN) it is natural to build an agent class that inherits from the DQN class (the alternative, namely to have a class-valued attribute in the agent class would be more appropriate if every agent had their individual DQN).

The core methods of our DQN class are the same as the functions in the Stage 2 Skeleton:

- `__init__(self, state_size, action_size = 4):` the previous `prepare_torch()` function naturally has moved into the initialization function. It does not need to be called separately but will be performed automatically as you initialise instances. It performs the entire initialisation that is required for PyTorch and generates a new pair of target and prediction networks.
- `get_qvals(self, state)`: this function returns the list of q-values for the state given as the argument from the prediction network.
- `get_maxQ(self, state)`: this function returns the maximal q-value for the state given as the argument from the target network.
- `update_target(self)`: this method copies the state of the prediction network to the target network.
- `train_one_step(self, states, actions, targets)`: this function performs a single training step. As arguments, it receives three parallel Python lists and the discount factor `gamma`. As before, the three lists together capture a minibatch for the update. Each triple `states[i], actions[i], targets[i]` fully describes one transition from the minibatch, with `states[i]` being the origin state, `action[i]` the chosen action and `targets[i]` the TD-target for this transition. Your program will have to compute the TD-targets based on the target state and the reward recorded in the replay buffer and pass them to this network training function.

These functions are documented in the notebook together with the open-source code. You don't need to worry about the source code if you are not interested.

You are welcome to program your own solution in Pytorch, Tensorflow, JAX/FLAX, MXnet[3] or any other neural network framework if you have a good working knowledge of one of these and want to do so. However, please be aware that

- This will incur an additional workload that cannot be accounted for in the FIT5226 assessment
- You will not receive additional or higher marks for this
- No additional help on this can be given by the teaching team (and indeed feedback on the assignment may be more limited since your teaching staff may not be experts in your chosen framework).

[3] Warning: The development of MXnet has now been discontinued by Apache. It may not be worth investing into learning this anymore. Vale MXnet!

Tasks for Stage 3.1 (Individual Component)

To solve these tasks, you are free to use any work that was produced by your own group in Stage 1 and Stage 2 and that you have participated in (but not that of any other groups).

1. **Contract:** Determine your "shopping list", ie. the list of options you want to use. This will constitute your "implementation contract." **This list must be submitted by Thursday, 3/10/2024.** it will be deemed final and binding upon submission. Your implementation will be assessed against this list, ie. you may not use any options not selected in your submission and you will be deemed to have used the options selected in your submission regardless of whether you ultimately use them in your implementation. Late submissions will not be accepted unless formal special consideration applies and **an unsubmitted list will be taken to mean "no options selected."** You are not allowed to confer with other students in the class about your choices before the submission, including your own group members. This would constitute collusion. Please fill in the contract form available in the Moodle Assignment section and submit it through the Moodle portal.
2. **Implementation:** Implement the Q-Learning scenario in Python as a Jupyter notebook using your chosen options. If you want to use any other additional libraries apart from the provided Skeleton, Numpy and Matplotlib check with your tutor beforehand whether these are admissible. Train and test your Q-Learning environment.To obtain performance points you must provide code so that the claimed performance can directly be tested.
3. **Documentation:** Devise a test procedure, and use appropriate metrics and visualisations to show that your agents learn the task successfully and that they learn to coordinate effectively. Document this in writing. Clearly explain how your training test procedure and metrics work.

Ideally, we would like the distance the two agents have to travel from their start position to the meeting point (which they can freely choose) to be fair, i.e. ideally both agents should travel the same distance. Analyse this possibility using game theory. What can you learn about the behaviour of the agents and useful rewards from a game theoretic analysis?

4. **(For bonus marks)[4] Game Theoretic Analysis:** Model this scenario as a game in the game-theoretic sense. To simplify matters, you are not required to use a 5x5 grid. Instead, you can use the smallest grid size that allows you to observe the relevant effects. Note that you will not be able to faithfully handle the spatial aspects of the scenario in a simple game theoretic analysis and you are not required to do so (i.e. imagine this as the agents being able to jump directly to a cell instead of having to walk there in several steps). Analyse the outcomes of your game. Based on your analysis, give an argument as to whether the agents' behaviour can be expected to converge on a fair solution and if so on which solutions it is expected to converge and why. Discuss the comparative merits of analysing the expected behaviour

[4] Bonus marks can be used to compensate for loss of points in other parts of this assignment or other assignments.

experimentally using the Q-learning implementation or theoretically using game theory. Are there any aspects that only one of these analyses can cover?

Tasks for Stage 3.2 (Group Component)

For the group component, you will perform a peer evaluation of the different solutions that the individuals in your group have developed. The results (marks) that you receive for the group component will not depend on how well the individual solutions function. It will only depend on the correctness and completeness of these solutions' comparative evaluation and on the insights that you gain from the comparison. Detailed instructions for Stage 3.2 will be published next week.

Submission Instructions

Detailed submission instructions will be published on Moodle in the Assignment section in Week 10.

Use of Generative AI

You are allowed to use Generative AI to solve your assignment. If you decide to do so, you must treat the AI like another external author (as a non-authoritative author whom you mistrust, given how much content is made up by Chat GPT and similar AIs). It is entirely your own responsibility that the content is correct and you can only use generated content to the extent that you could use materials provided by an external author. The AI is not part of your project team. If you use code that an AI produced you must be able to fully explain every detail of that code.

Details will be given with the individual assessment items.

Any use of generative AI must be appropriately acknowledged (see Learn HQ).